Triggers

Triggers are somewhat like stored procedures except that they are activated automatically. When is a trigger activated? A trigger is activated on the occurrence of a particular event. What are these events that can cause the activation of triggers?

These events may be database update operations like INSERT, UPDATE, DELETE etc. A trigger consists of these essential components:

• An event that causes its automatic activation.

• The condition that determines whether the event has called an exception.

• The action that is to be performed.

Triggers do not take parameters and are activated automatically, thus, are different to stored procedures on these accounts. Triggers are used to implement constraints among more than one table. Specifically, the triggers should be used to implement the constraints that are not implementable using referential integrity or constraints. An instance of such a situation may be when an update in one relation affects a few tuples in another relation. However, please note that you should not be over-enthusiastic for writing triggers – if any constraint is implementable using declarative constraints such as PRIMARY KEY, UNIQUE, NOT NULL, CHECK, FOREIGN KEY, etc., then it should be implemented using those declarative constraints rather than triggers, primarily due to performance reasons.

You may write triggers that may execute once for each row in a transaction – called Row Level Triggers, or once for the entire transaction - called Statement Level Triggers. Remember that you must have proper access rights on the tables on which you are creating the trigger. The following is the syntax of triggers in one of the commercial DBMSs:

CREATE TRIGGER <trigger_name>

[BEFORE | AFTER|

<Event>

ON <tablename>

[WHEN <condition> | FOR EACH ROW]

<Declarations of variables, if needed, – may be used when creating trigger

using host language>

BEGIN

<SQL statements OR host language SQL statements>

[EXCEPTION]

<Exceptions if any>

END;

Let us explain the use of triggers with the help of an example:

Example 8: Consider the following relation of a student's database:

STUDENT (enrolno, name, phone)

RESULT (enrolno, coursecode, marks)

COURSE (course-code, c-name, details)

Assume that the marks are out of 100 in each course. The passing marks in a subject are 50. The University has a provision for 2% grace marks for the students who are failing marginally – that is, if a student has 48 marks, s/he is given 2 marks grace and if a student has 49 marks, then s/he is given 1 grace mark. Write the suitable trigger for this situation.

Please note the requirements of the trigger:

Event: UPDATE of marks

OR

INSERT of marks

Condition: When a student has 48 OR 49 marks

Action: Give 2 grace marks to the student having 48 marks and 1 grace mark to the student having 49 marks.

The trigger for this thus can be written as:

```
CREATE TRIGGER grace
AFTER INSERT OR UPDATE OF marks ON RESULT
WHEN (marks = 48 OR marks =49)
UPDATE RESULT
SET marks =50;
```

We can drop a trigger using a DROP TRIGGER statement.

DROP TRIGGER trigger_name;

The triggers are implemented in many commercial DBMSs.

Exercises: Write a trigger that restricts updating of the STUDENT table outside the normal working hours/holiday.

10. TRIGGERS AND ACTIVE DATABASES:(*****)

→A **trigger** is a procedure that is automatically invoked by the DBMS in response to specified changes to the database, and is typically specified by the DBA.
→A database that has a set of associated triggers is called an **active database.**
→A **trigger** description contains three parts:

1. **Event**
2. **Condition**
3. **Action**

1. **Event:** A change to the database that activates the trigger.

→Event describes the modifications done to the database which lead to the activation of trigger. The following are fall under the category of events,

i) Inserting, updating, deleting columns of the tables or rows of tables may activate the trigger.
ii) Creating, altering or dropping any database object may also lead to activation of triggers.
iii) An error message or user log-on or log-off may also activate the trigger.

2. **Condition:** A query or test that is run when the trigger is activated.

→Conditions are used to specify whether the particular action must be performed or not. If the condition is evaluated to true then the respective action is taken otherwise the action is rejected.

3. **Action:** A procedure that is executed when the trigger is activated and its condition is true.

→The examples shown in **Figure 5.19**
→The trigger called **init_count** initializes a counter variable before every execution of an INSERT statement that adds tuples to the Students relation.
→The trigger called **incr_count** increments the counter for each inserted tuple that satisfies the condition age < 18.

SRCR RCOE

```
CREATE TRIGGER init_count BEFORE INSERT ON Students          /* Event */
    DECLARE
        count INTEGER;
    BEGIN                                                    /* Action */
        count := 0;
    END

CREATE TRIGGER incr_count AFTER INSERT ON Students           /* Event */
    WHEN (new.age < 18)          /* Condition: 'new' is just-inserted tuple */
    FOR EACH ROW
    BEGIN                /* Action; a procedure in Oracle's PL/SQL syntax */
        count := count + 1;
    END
```

Figure 5.19 Examples Illustrating Triggers

→A **row-level trigger** is activated for each modified record, a **statement-level trigger** is activated only once per INSERT command.

Advantages of trigger:

1) Triggers can be used as an alternative method for implementing referential integrity constraints.

2) By using triggers, business rules and transactions are easy to store in database and can be used consistently even if there are future updates to the database.

3) It controls on which updates are allowed in a database.

4) When a change happens in a database a trigger can adjust the change to the entire database.

5) Triggers are used for calling stored procedures.

Introduction to Triggers in SQL

- `CREATE TRIGGER` statement
  - Used to monitor the database
- Typical trigger has three components which make it a rule for an "active database " (more on active databases in section 26.1) :
  - **Event(s)**
  - **Condition**
  - **Action**

USE OF TRIGGERS

- AN EXAMPLE with standard Syntax.(Note : other SQL implementations like PostgreSQL use a different syntax.)

R5:
CREATE TRIGGER SALARY_VIOLATION
BEFORE INSERT OR UPDATE OF Salary, Supervisor_ssn **ON** EMPLOYEE

FOR EACH ROW
WHEN (NEW.SALARY > **(** SELECT Salary FROM EMPLOYEE
WHERE Ssn = NEW. Supervisor_Ssn))
INFORM_SUPERVISOR (NEW.Supervisor.Ssn, New.Ssn)

Note: Refer to the lab assignments on Triggers for various types of triggers.